

PMS-VECTOR – Model Specification

Direction and Rotation: a PMS-bound reference model for bounded structural orientation, perspectival pressure, and reducible claim architecture

Version: PMS-VECTOR_1.0 · Spec basis: PMS-VECTOR.yaml

Author: T. Zöller · Formalisation assistance: ChatGPT (GPT-5.6 Sol)

Language: EN · Status: Model spec (aligned with schema_meta.status = "draft_repository_reference", schema_meta.maturity = "paper_complete_repository_formalization_in_progress")

Depends on: PMS.yaml (schema_version = "PMS_1.3") · Theoretical source: PMS-VECTOR.md

1. Purpose and scope of this specification

This document specifies the repository model of *PMS-VECTOR* in a concise, technical form. It is based on `PMS-VECTOR.yaml` (with `schema_version = "PMS-VECTOR_1.0"`) and explains how the machine-readable profile is to be read together with the governing PMS Base specification and the theoretical paper `PMS-VECTOR.md`. It is a human-readable interpretation and field contract for the repository model; it is not an additional source of VECTOR theory.

PMS-VECTOR is a **late meta-reconstructive praxeological orientation theory**. Its central task is to reconstruct when already structured differences may support bounded directional claims, how those claims behave under admissible perspectival pressure, and under what conditions they must be narrowed, suspended, reduced, or left outside VECTOR. The model remains explicitly revisable, rival-sensitive, and non-final. It does not supply a universal rank, a general moral decision rule, a person-ranking system, or a default replacement for specialist methods.

The repository profile is intentionally positioned between two errors. It is **not a free-standing schema detached from PMS**, because its PMS operator references and PMS-conformity claims require the governing semantics of `PMS.yaml`. It is also **not a thin alias for PMS**: VECTOR owns its own claim architecture, status distinctions, result types, reduction conditions, and repository interfaces. The dependency is therefore asymmetric: VECTOR-specific constructs remain visible and inspectable in this file, while PMS Base semantics are resolved externally rather than duplicated.

This specification covers, in particular:

- the **hard semantic dependency on PMS_1.3** and the conditions under which a PMS-VECTOR application is or is not valid;
- the **typed authority structure** separating PMS Base semantics, VECTOR theory, machine-readable representation, and downstream repository artifacts;
- the **VECTOR identity and construct-status system**, including the distinction between VECTOR-owned constructs, PMS references, stored audit state, derived supporting constructs, external warrant inputs, and reduction statuses;
- the **claim-bearing reference coordinates** $(0, F, Q, \pi, t)$ and the rule that there is no context-free VECTOR state;
- the **application, authority, human, calibration, and geometry boundaries** that constrain what a formal VECTOR record may imply;
- in later sections, the core DIR-ROT architecture, stored ROT profile, derived DIST, result types, option and agency relations, consequence and attribution interfaces, test burden, rival superiority, self-reduction, record structures, provenance, computing boundaries, and Reader integration.

Authority boundary of this specification

`PMS-VECTOR.yaml` is not a standalone replacement for `PMS.yaml`, and this Model Specification is not an additional source of VECTOR theory. PMS Base semantics remain governed by `PMS.yaml`; VECTOR-specific theoretical claims remain governed by `PMS-VECTOR.md`; the YAML is their machine-readable repository mirror.

Methodological restraint

PMS dependency resolution does not validate VECTOR theory. Machine-readable consistency does not establish theoretical adequacy. Formal explicitness does not establish metric validity. Repository completeness, case coverage, or Reader consistency cannot by themselves supply external warrant or rival superiority.

2. Authority, PMS dependency, and repository role

2.1 Hard semantic dependency on PMS

PMS-VECTOR is built in explicit dependency on the Praxeological Meta-Structure. The dependency is encoded by `pms_dependency_gate` and is **semantic rather than merely bibliographic**. A repository tool may inspect VECTOR-specific keys without loading the Base specification, but it may not thereby claim a valid PMS-VECTOR application, complete PMS operator semantics, or PMS-conformity.

Dependency rule	Repository meaning	Consequence if PMS is unresolved
<code>source_file: PMS.yaml</code>	Governing Base specification	VECTOR may be inspected, but not treated as a complete PMS application profile.
<code>expected_schema_version: PMS_1.3</code>	Expected Base compatibility target	Version mismatch must be treated as unresolved compatibility, not silently ignored.
<code>resolution_required: true</code>	PMS semantics must be resolved for valid application	<code>pms_dependency_unresolved</code>
<code>vector_specific_constructs_visible_without_pms: true</code>	VECTOR retains its own inspectable identity	Inspection does not become PMS-conform application.

The required Base bindings are intentionally narrow. VECTOR resolves the PMS entry condition, repository-governance boundary, meta-axiom identity and dependency semantics, and the Base computational restraint from `PMS.yaml`. PMS-derived structures are conditional references only where a particular VECTOR claim explicitly invokes them; they are not universal runtime dependencies of every VECTOR analysis.

Unresolved-dependency rule
Without a resolved `PMS.yaml`, a consumer may not infer PMS operator dependencies from this file, treat local operator labels as self-contained definitions, claim PMS-conformity, or use PMS-VECTOR as a replacement Base grammar. Criticism, rejection, revision proposals, and rival formalizations remain possible outside this application gate.

2.2 PMS references are handles, not redefinitions

The `pms_reference` block lists the fixed operator handles Δ - Ψ so that VECTOR can make explicit cross-references. Their canonical definitions, order, dependency relations, and Base semantics remain external. The profile therefore records operator identity without reproducing a second PMS grammar.

VECTOR-side reference practice	Permitted?	Reason
Reference Δ - Ψ by fixed handle	Yes	Necessary for explicit lineage and type boundaries.
Duplicate PMS operator definitions as VECTOR-owned semantics	No	Would detach the overlay from the governing Base and risk semantic drift.
Rewrite the PMS operator dependency path	No	VECTOR is a later comparative theory, not a Base revision.
Use PMS openness as proof of VECTOR	No	Base openness permits inquiry; it does not warrant VECTOR conclusions.

2.3 Typed authority

Repository authority is **typed rather than globally hierarchical**. Different artifacts govern different semantic questions. This prevents both Base semantic drift and the opposite error of treating VECTOR theory as if it were merely subordinate documentation of PMS.

Authority domain	Authoritative artifact	Governs
PMS Base semantics	PMS.yaml	PMS entry condition, operator identity and semantics, dependency path, Base/overlay governance
VECTOR theory	PMS-VECTOR.md	VECTOR-specific theoretical claims, boundaries, reductions, and paper-linked burden
VECTOR machine representation	model/PMS-VECTOR.yaml	Machine-readable mirror, status contracts, references, and repository integration
Model Specification	This document	Human-readable interpretation and field contract; no additional theory authority
Downstream artifacts	Reference files, cases, examples, Reader	Derived representation, test records, illustration, presentation, and repository-consistency support

This typed structure is part of VECTOR's non-inflation discipline. A downstream artifact may become more explicit or easier to inspect without becoming more authoritative than its source. In particular, schema-valid records do not create warranted VECTOR results, case count does not become evidence weight, and Reader graph relations do not create theory.

2.4 Claim type and ecosystem position

The repository model identifies PMS-VECTOR as a **late meta-reconstructive praxeological orientation profile**. Each term constrains the claim type rather than elevating it.

Term	Meaning in VECTOR	Does not imply
Late	Posterior to the local structures whose relations are reconstructed	Final, higher, more mature, or historically necessary
Meta-reconstructive	Reconstructs relations among already reconstructed structures	A new ontological layer or automatic meta-level authority
Praxeological	Remains bound to situated praxis, frames, asymmetry, temporality, possibility, consequence, and capacity	A general metaphysics of value or person ranking
Orientation	Asks whether bounded structural differences carry direction under declared conditions	Prescription, mandate, or universal ranking

Orientation is not authority
Better orientation may increase answerability within a declared scope without increasing entitlement to decide for others. In model terms: `Orientation↑` does not imply `Authority↑`. Law, consent, role, institution, expertise, mandate, moral premises, or other external conditions remain separate possible sources of authority.

3. High-level structure and construct status system

3.1 Model identity

VECTOR's machine-readable identity is compact by design. The canonical theory core is `VECTOR := (DIR, ROT)`. A concrete typed state is represented as `V_t(0 | F, Q, π) = (DIR_t, RP_t)`. The two expressions are not interchangeable: the first names two function families; the second records current claim-relevant state. The detailed DIR, ROT, and RP contracts are specified in Sections 5–7 of this document.

Element	Status family	Repository meaning
DIR	vector_core_claim_family	VECTOR-owned directional claim family
ROT	vector_core_operation_family	VECTOR-owned perspective-testing and discovery operation family
RP	vector_stored_audit_state	Stored rotation-test profile; not the ROT operation itself
DIST	vector_derived_supporting_construct	Derived descriptor of effective orientation-enactment separation; not a core component
AdultOrientation	vector_supporting_shorthand	Supporting shorthand only; no independent theoretical discrimination

The current machine-readable theory status also records actual reductions already incurred by the paper. DIR and ROT are retained with narrowed scope; DIST is derived supporting only; Adult Orientation is supporting shorthand only; and standalone VECTOR is provisionally retained with scope reduction. These statuses are not decorative maturity labels. They are part of the theory's explicit self-reduction state and must not be silently promoted by downstream tooling.

3.2 Construct-status registry

The `construct_status_registry` prevents category collapse among PMS references, VECTOR-owned constructs, stored state, result types, external inputs, provenance labels, and reduction states. Consumers should treat an unknown status as unresolved rather than infer a nearest category.

Status family	Ownership	Examples / role
pms_operator_reference	Not VECTOR-owned	$\Delta, \nabla, \square, \wedge, A, \Omega, \Theta, \Phi, X, \Sigma, \Psi$; semantics from PMS.yaml
pms_derived_reference	Not VECTOR-owned	Conditional Base-derived references where a claim explicitly invokes them
vector_core_claim_family	VECTOR-owned	DIR
vector_core_operation_family	VECTOR-owned	ROT
vector_stored_audit_state	VECTOR-owned	RP
vector_derived_supporting_construct	VECTOR-owned, non-core	DIST
vector_supporting_shorthand	VECTOR-owned shorthand	AdultOrientation; no independent theoretical discrimination
vector_result_type	VECTOR-owned	Typed result vocabulary; detailed in Section 7
external_warrant_input	External	Historical evidence, measurement, current law, domain knowledge, causal support, authority evidence
external_calibration_input	External	Threshold bases, criterion weights, specialist scales
provenance_status	VECTOR-owned label system	C / R / N / P; origin/status rather than epistemic strength

Status family	Ownership	Examples / role
reduction_status	VECTOR-owned status family	Retain, narrowed, supporting-only, scope-reduced, non-capture, architecture-reduced, and related states

Do not collapse status families

A PMS operator reference is not a VECTOR component. A stored state is not an operation. A derived supporting construct is not a core primitive. An external warrant input is not generated by VECTOR. A provenance label is not an evidence grade. A reduction status is not a person or case classification.

3.3 High-level YAML organization

The YAML is organized by theoretical responsibility rather than by one universal PMS-addon template. The major groups are shown below so that software and human readers can locate the relevant contract without treating file order as a theory hierarchy.

Model area	Primary keys	Function
Identity and governance	pms_dependency_gate , schema_meta , repository_governance , pms_reference	Authority, Base dependency, repository role, methodological restraint
Type and application ground	vector_identity , construct_status_registry , validity_and_application_boundary , geometry_firewall , reference_coordinates , calibration_boundary	Claim type, status families, coordinates, boundaries, calibration dependence
Core and supporting theory	vector_architecture , dir , rot , rot_profile , dist_derived , reversibility , vector_dynamics	Core functions, stored audit state, derived separation, dynamics
Praxis relations	option_space , second_order_agency , consequence_and_cost_topology , timing , orientation_uptake , structural_attributability , ought_typology , constraint_architecture	Possibility, agency, consequence, uptake, attribution, ought, constraint
Test and reduction architecture	convergence , bounded_test_burden_closure , failure_families , falsifiability_and_reduction , rival_superiority , current_reduction_status , non_capture	Pressure, failure, rivalry, self-reduction, scope loss
Repository interfaces	record_reference , case_suite_reference , provenance_reference , dependency_reference , computational_representation , reader_boundary	Downstream records, cases, references, machine representation, presentation boundary

4. Reference coordinates, validity, calibration, and geometry firewall

4.1 No context-free VECTOR state

VECTOR claims are constituted by a minimum set of claim-bearing coordinates rather than attached to a context-free value. The canonical coordinate tuple is $(0, F, Q, \pi, t)$, where the reference object, frame, question and claim scope, perspective, and temporal position remain visible enough to determine what claim is being made.

Coordinate	Meaning	Minimum declaration	Type boundary
0	Reference object	Typed referent: e.g. praxis scene, reconstruction, trajectory, coordination relation, option field, claim, institutional arrangement	$0 \neq \text{person worth}$
F	Declared frame	Included and excluded boundaries	$F \neq \square$ (PMS Frame operator)
Q	Question and claim scope	Question scope and claim ceiling	Question change may create a new bounded claim
π	Perspectival position	Declared viewing / relational position	$\pi \neq P_MIP$
t	Temporal position or evaluation index	Temporal position; scene/analysis split where material	$t \neq \emptyset$

These coordinates are not optional descriptive metadata. They determine the identity and scope of the claim. Material change in one or more coordinates creates a new continuity burden; the model does not assume that a claim automatically survives a changed reference, frame, question, perspective, or temporal position.

Temporal anti-retrojection

Where retrospective analysis makes scene time and analysis time materially distinct, use t_s and t_a . Analyst legibility at t_a does not imply actor legibility at t_s , and later knowledge does not imply effective capacity at scene time. The split is an anti-retrojective aid, not an expansion of the VECTOR core tuple.

4.2 Typed, bounded, and revisable application

A valid VECTOR claim must remain typed, bounded, and revisable. The YAML expresses this as a set of application requirements rather than as a guarantee that the resulting claim is substantively correct.

Requirement family	Repository contract
Typing	Reference type, frame, question, perspective, temporal position, and relevant claim type remain explicit.
Boundedness	Reference, frame, question/claim ceiling, perspective, and temporal scope constrain what the result may say.
Revisability	The claim remains exposed to weakening, trade-off, reversal, narrower scope, suspension, or non-capture.
Warrant visibility	Material external warrant sources and dependencies remain visible rather than being generated by notation.
Calibration visibility	Threshold bases are declared where material; VECTOR does not manufacture all substantive thresholds internally.

Boundedness is part of claim content rather than a disclaimer attached after the fact. Widening a frame, changing a question, extending a temporal interval, or generalizing a local result creates a new burden. Likewise, structural orientation and practical effect do not automatically create authority or human rank.

Application non-equivalences

Legibility is not direction. Direction is not authority. Orientation effect is not orientation truth. Structural robustness is not practical attribution. Being told is not uptake. Visibility is not effective capacity. A system-level carriage requirement does not by itself establish that a particular person must carry it.

4.3 Calibration boundary

VECTOR can identify and structure thresholds on which an orientation claim depends, but it does not generate all such thresholds endogenously. Terms such as *relevant*, *materially distinct*, *sufficient*, *effective*, *actionable*, *viable*, *claim-critical*, and *robust* may require calibration from source material, empirical measurement, domain convention, legal or technical standards, role-specific conditions, explicit comparison criteria, or another externally accountable basis.

Calibration property	VECTOR rule
Threshold exposure	Material thresholds and their dependencies should be visible and revisable.
Endogenous generation	VECTOR does not claim to generate all substantive thresholds from its own vocabulary.
Conform divergence	Two VECTOR-conform analyses may remain calibration-divergent.
Permitted response	Narrower scope, explicit divergence, bounded orientation, Suspend, rival-sensitive disagreement, or Non-Capture.
Failure boundary	Failure begins when material threshold divergence becomes hidden, untraceable, or is represented as if VECTOR itself had supplied the substantive calibration.

Calibration is not warrant by notation

A typed threshold is not thereby a justified threshold. A symbol, schema field, or internally consistent record can expose calibration dependence without resolving it. Calibration remains external to the VECTOR core unless a bounded domain separately supplies it.

4.4 Geometry firewall and non-metric default

The vocabulary of *vector*, *direction*, *rotation*, *distance*, and *resultant* is relational and ordinal before metric. Unless a bounded domain separately earns stronger mathematics, the notation must not be read as importing Euclidean vector-space properties.

Expression	Required reading
VECTOR	Not a vector space by default
DIR	Not a signed scalar; not identical to PMS ∇
ROT	Not geometric rotation; not identical to PMS Φ ; not STRATA PROJECT_AS
DIST	Not metric distance; not identical to PMS X; not a VECTOR core component
DIR_R	Not a vector sum or global scalar
π	Perspective coordinate; not MIP practical power P_MIP
t	Temporal index; not PMS Θ

No norm, magnitude, angle, dot product, vector addition, Euclidean projection, default unit for DIST, invertibility of ROT, group composition of ROT, total ordering, or transitivity across unrelated questions follows from the notation. A domain-specific metric may be added only as an additional bounded claim with explicit measurement target, calibration, uncertainty, meaning-preserving transformations, and non-generalization boundaries.

Formal representation is not metric validation

VECTOR may represent a bounded directional relation without assigning a universal score. The absence of a universal scale does not imply the absence of orientation, and the absence of total order does not imply the absence of comparative discrimination. Quantification, where warranted by a domain, is an additional claim rather than a default property of the framework.

Package I stop condition

At this point the reader should know what the model is, which artifact governs which semantics, why PMS is a hard dependency without absorbing VECTOR's identity, which construct/status classes exist, which coordinates make a VECTOR claim well-formed, and which application, calibration, authority, human, and geometry boundaries constrain all later sections. The detailed mechanics of DIR, ROT, RP, derived DIST, reversibility, and result typing begin in Package II.

5. Core VECTOR architecture: DIR, ROT, and RP

The current VECTOR core is deliberately compact. It consists of the bounded directional claim family DIR (Direction) and the perspectival testing and discovery operation family ROT (Rotation):

Canonical core identity

`VECTOR := {DIR, ROT}`

The core identity is not the same thing as a concrete stored VECTOR state. At a declared temporal position, the repository profile represents the current state as:

Concrete VECTOR state

`V_t(0 | F,Q,π) = {DIR_t, RP_t}`, where `RP_t := ROTProfile_t` records the current rotation-test profile. `ROT` is the operation family; `RP` is stored audit state. Therefore `ROT != RP`.

This distinction prevents a category drift that would otherwise be easy to introduce in machine-readable use. The core says what kinds of VECTOR work exist; the state says what a bounded analysis currently records. Support fields such as warrant, calibration, option space, effective capacity, consequence relations, provenance, residue, or result status may condition a VECTOR claim without becoming additional VECTOR components.

5.1 DIR (Direction): bounded directional claim family

DIR names a locally earned directional relation under declared comparison conditions. It begins from the distinction that structural difference does not automatically produce direction, and that direction does not automatically produce total order. A DIR claim therefore states only what the declared reference, frame, question, perspective, temporal position, comparison basis, warrant, and claim ceiling support.

DIR property	Repository contract
Status	<code>vector_core_claim_family</code>
Scope	Local, reference-bounded, frame-bounded, question-bounded, perspective- and time-indexed
Comparison character	Relational and ordinal before metric; no default magnitude or universal scalar
Compensation	Non-compensatory by default where claim-critical dimensions are not legitimately commensurable
Warrant	Must remain traceable to declared sources; VECTOR does not manufacture external warrant
Claim ceiling	Directional wording may not silently expand beyond the scope that earned it
Rival vulnerability	A same-burden rival may narrow or displace VECTOR's directional role

5.2 Canonical DIR relations

Let $\mathbf{c} = (0, F, Q, \pi, \tau)$ collect the claim-bearing VECTOR conditions. A directed relation is represented canonically as:

Directed relation

$X \prec_{\{\mathbf{V}, \mathbf{C}\}} Y$

The notation means that the bounded comparison burden defined by $\mathbf{c}$ supports direction from X toward Y . It does not mean that Y is universally better, morally superior, greater on an underlying universal scale, or preferred for every question. The symbol is directional notation before it is an order-theoretic commitment.

In particular, transitivity is not assumed by default. Even where two local relations are individually warranted, their conditions may differ or the relevant domain may not support a transitive subrelation. No repository consumer may therefore infer a global order merely from repeated use of $\prec$.

Relation	Canonical notation	Meaning
Directed result	$X \prec_{\{\mathbf{V}, \mathbf{C}\}} Y$	A bounded non-compensatory comparison supports direction.
Trade-off / multidirectional result	$\text{TradeOff}_V(X, Y \mid \mathbf{c})$	Several relevant directions remain visible, but no legitimate single resultant is warranted.
Incomparability	$X \parallel_{\{\mathbf{V}, \mathbf{C}\}} Y$	A meaningful common comparison exists, but no warranted dominance relation has been earned.
Non-Capture	$\emptyset_{\{\mathbf{V}, \mathbf{C}\}}$	The directional operation should not be asserted in the current form.

DIR non-equivalences

Difference is not direction; direction is not total order. Incomparability is not equality. Trade-Off is not Incomparability. Incomparability is not Non-Capture. Local superiority does not imply universal superiority. A repeated local direction does not create an ecosystem-wide scalar merely by recurrence.

5.3 DIR validity conditions and non-compensation

The repository profile treats a serious DIR claim as valid only when its comparison burden is visible enough to reconstruct. The YAML therefore records conditions rather than a numeric scoring rule.

- **Reference continuity:** the comparison must remain about the relevant object or relation.
- **Declared comparison basis:** the basis carrying direction must be stated rather than implied.
- **Warrant visibility:** source support and unresolved external premises remain inspectable.
- **Question-bounded relevance:** the alleged directional difference must matter for the declared question.
- **Non-compensation:** claim-critical losses cannot be hidden by unexplained aggregate gain.
- **Calibration visibility:** thresholds or priorities that matter to the result must be exposed.
- **Scope-bounded claim ceiling:** stronger wording requires a stronger burden.
- **Rival vulnerability:** the claim must remain exposed to serious same-burden alternatives.

Non-compensation is the default because many VECTOR comparisons involve dimensions that have not been shown to share a legitimate common scale. A trade-off may be represented, but one loss may not disappear into an aggregate score unless commensurability and calibration are separately warranted. VECTOR therefore permits local ranking without requiring universal commensuration.

5.4 DIR failure and reduction condition

DIR is not protected merely because it is a named core component. The YAML preserves the paper's self-reduction condition: DIR must narrow or lose standalone discrimination where relevant cases can be captured losslessly by pre-existing structures or where a same-burden rival provides the required

directional discrimination more directly. The current reduction consequences are stated later in this specification; the important architectural point here is that core status does not imply immunity from reduction.

5.5 ROT (Rotation): perspectival testing and discovery operation family

ROT is the second VECTOR core component. It applies an admissible perspectival change in order to test, damage, revise, split, collapse, or extend a bounded directional claim. ROT is therefore not simply the act of looking again. For robustness credit, the rotation must expose a credible route by which the current claim could lose something material.

Canonical ROT operation
ROT_p(C₀) → C₁

The symbol *p* identifies the declared rotation relation or perspective change. C₀ is the pre-rotation claim condition; C₁ is the post-rotation condition or result. This is an operation signature, not geometric rotation, and it carries no implication of angle, invertibility, group structure, or spatial coordinate transformation.

5.6 Robustness-bearing ROT versus exploratory probe

The model distinguishes a robustness-bearing rotation from an exploratory probe. Both can be useful, but they do not earn the same claim status.

Requirement	Robustness-bearing ROT	Exploratory probe
Declared change	Required	Required enough to make the exploratory move legible
Declared purpose / vulnerability	Required	May be discovery-oriented rather than loss-testing
Held constants	Required for the relevant same-claim test	May change where the probe deliberately opens a new frame
Reference continuity	Required where survival of the same claim is asserted	May be relaxed if the output is treated as a new probe rather than survival evidence
Question relevance	Required	May produce a new question instead
Material distinctness	Required	Desirable but not sufficient for robustness credit
Credible claim-loss route	Required	Not required for discovery, therefore no robustness credit by itself

Rotation must be able to hurt
A ROT operation that cannot plausibly damage, narrow, split, revise, suspend, or collapse the claim may still be useful as exploration, but it does not count as robustness-bearing pressure. Discovery does not automatically become warrant, and a new distinction produced by ROT must itself be typed, bounded, and tested.

5.7 Rotation families

The YAML enumerates recurrent rotation families. These are named pressure patterns rather than exhaustive ontological categories. A concrete analysis may use one or several where they are materially relevant to the declared vulnerability.

Rotation family	Primary pressure question
Role Rotation	Does the directional claim survive a materially different role position?
Actor / Affected-Role Rotation	Does the claim change when the actor's perspective is separated from the affected role?

Rotation family	Primary pressure question
Inside / Outside Rotation	Does an internal account survive a relevant external view, or vice versa?
Intention / Effect Rotation	Does a claim grounded in intention survive pressure from effects, or the reverse?
Temporal Rotation	Does the claim survive a materially different temporal position or retrospective/prospective cut?
Ω -Position Rotation	Does the claim survive a materially different asymmetry position?
Counterfactual Rotation	Does a relevant counter-scene expose hidden dependence or loss?
Rival-Model Rotation	Can a serious rival capture the same burden more adequately or more directly?
Granularity Rotation	Does the claim survive a materially different level of structural resolution?

5.8 ROT outcomes and admissible stopping

A rotation record may classify its immediate relation to the tested claim as `invariant`, `variant`, `emergent`, or `collapse`. These labels describe the effect of the pressure; they do not themselves replace the final result types defined in Section 7.

ROT is also bounded by an admissible stop rule. Testing need not continue indefinitely merely because additional imaginable perspectives exist. A current test may stop where known claim-critical vulnerabilities have received material pressure, remaining rotations are redundant or irrelevant, further testing requires unavailable external evidence, further pressure belongs to a stronger new claim, or the current result is already `Suspend`, `Non-Capture`, or `Revision Required`. The stop is a bounded test decision, not finality.

ROT reduction condition

ROT must reduce if, under the same burden, it cannot add claim-relevant discrimination beyond ordinary reframing, rival comparison, or already available PMS recontextualization practices. The framework therefore cannot defend ROT by counting every perspective change as a unique VECTOR contribution.

5.9 RP (ROT Profile): stored audit state

`RP` stores what has happened under ROT pressure. It is not an operation and is not a third VECTOR core component. Its purpose is to preserve an auditable test profile so that a current state can distinguish untested confidence from pressure-exposed orientation.

RP field family	What is recorded
Rotation identity	Rotation family, purpose, and targeted vulnerability
Transformation record	Changed variables, held constants, reference continuity, and material distinctness
Loss architecture	Credible loss route and the result of applying pressure
Robustness relevance	Whether the pressure bears on the current claim's robustness status
Residual burden	Untested relevant rotations and the current stop basis

Core / state firewall

`VECTOR` := `(DIR, ROT)` describes the current core. `Vt` = `(DIRt, RPt)` describes a stored state. `ROT` is performed; `RP` is recorded. Warrant, calibration, option space, cost topology, uptake, provenance, result status, residue, and derived `DIST` may be indispensable to a claim without becoming VECTOR core components.

6. Derived DIST, reversibility, and VECTOR dynamics

DIST (Distance) remains part of VECTOR's descriptive vocabulary, but its current theoretical status is explicitly reduced. It is not a third core component and not an independent causal dial. The repository model retains it only as a derived supporting descriptor of effective orientation–enactment separation carried by more specific source relations.

Current DIST status
`DIST = derived orientation–enactment separation descriptor; in_vector_core: false ;
independent_component: false ; default_metric: false .`

6.1 Source relations of derived DIST

Effective orientation–enactment separation is constituted by interacting structural and practical relations rather than one universal variable. The YAML records recurrent sources from the PMS ecosystem and explicit non-operator conditions so that DIST remains traceable to the relations that actually carry discrimination.

Source family	Declared sources
PMS operator references	X, Θ, Ψ, Ω, A
Explicit non-operator sources	publicness, option status, effective capacity, reversibility conditions, role position

These references do not turn DIST into a sum of operators. Nor does any one source relation determine DIST universally. The repository contract is instead causal and evidential restraint: strong claims must be carried by the underlying relations that generate effective separation. DIST may summarize the relation after those sources are stated, but it may not substitute for them.

DIST discrimination rule
Source relations carry the strong claim. DIST may summarize them. DIST may not operate as an independent warrant source, independent causal mechanism, universal quality axis, or free-standing scalar. Its reduction basis is the paper's DIST necessity test and current architecture reduction.

6.2 Orientation–enactment separation is practical, not geometric

DIST tracks the practical relation between current orientation and consequential consolidation. It can summarize how much meaningful revision, interruption, reconsideration, reframing, or redirection remains before further binding changes the field. This does not create a geometric notion of nearness, a default unit, or a formula connecting DIST to speed of enactment.

A configuration may exhibit narrow effective separation because deadlines, public commitments, resource commitments, institutional sequencing, dependency, or prior binding have reduced the practical revision interval. Another may retain wide separation because correction, pause, exit, or reframing remain readily available. The direction of quality does not follow from this difference.

No monotonic DIST rule
`DIST↑` does not imply better orientation or higher adequacy. Low derived DIST can indicate premature consolidation or a genuinely closing field. High derived DIST can preserve correction or become excessive analytical delay. The relevant assessment remains local to the declared question, timing, capacity, binding, asymmetry, and consequence structure.

6.3 Role-relative separation

The profile does not require one scene-wide DIST value. Different actors or roles may possess materially different revision possibilities in the same nominal situation. One role may still be exploring while another has already absorbed costs, binding, dependency, or loss. Such differences are not merely perspective differences; they may be differences in actual practical possibility.

This role-relativity is another reason not to treat DIST as a universal scalar. Where a bounded domain legitimately compares separation relations, the underlying role and source relations must remain visible enough to show what the comparison means.

6.4 Reversibility: a distinct path-sensitive dependency architecture

Present orientation–enactment separation does not reveal what prior praxis has already made costly, unavailable, or impossible to restore. VECTOR therefore keeps reversibility separate from DIST. The supporting relation `REV_avail(a, t | F, Q)` records the practically meaningful correction, redirection, exit, modification, or partial recovery still available to a relevant actor or role under bounded conditions.

Reversibility firewall
`DIST ≠ REV_avail · reversibility ≠ restoration · C_ROT ≠ C_REV`

Concept	Repository meaning
Derived DIST	Current effective orientation–enactment separation
<code>REV_avail</code>	Path-sensitive remaining range of practical correction, redirection, exit, modification, or partial recovery
Restoration	Recreation of a prior state; not required for meaningful reversibility
<code>C_ROT</code>	Practical friction involved in reorienting a relevant actor, role, institution, or interpretive field
<code>C_REV</code>	Cost of materially changing enacted or bound praxis

The distinction between rotation cost and reversal cost is important. The former is not the analyst's computational effort in performing a ROT operation; it concerns practical reorientation in the relevant social or institutional field. The latter concerns what it costs to alter a path that has already become enacted or bound. A claim may be easy to rethink while the corresponding praxis remains expensive to reverse.

6.5 Path dependence and non-restorative correction

Reversibility is path-dependent, role-relative, temporally conditioned, affected by binding and public fixation, and potentially asymmetric. Two present states can look similar while carrying different histories of commitment, lost options, downstream effects, or exposure; those histories may produce different remaining correction relations.

Exit therefore changes trajectory without resetting it. Renewed orientation–enactment separation does not undo temporal sedimentation. ROT does not erase prior self-binding. A reopened revision interval does not restore lost history. The model preserves room for correction without pretending that correction recreates a pre-enactment state.

No universal reversibility optimum
More available reversibility is not universally better, and `REV_avail↑` does not imply higher praxis quality. Reversibility may preserve correction, but indefinite avoidance of binding can itself destroy options, miss windows, or externalize burdens. The judgment remains question- and trajectory-bounded.

6.6 VECTOR dynamics remain open and recursive

The YAML intentionally does not specify a closed update law for VECTOR. The model records supported dependency relations while leaving external events, actors, evidence, and structural change open. VECTOR therefore does not claim a complete state-transition engine for praxis.

Supported dynamic relation	Permitted interpretation
ROT may change DIR	Perspectival pressure can weaken, split, revise, collapse, or extend a directional claim.
Source conditions may change practical availability of ROT	What remains analytically conceivable need not remain practically available to an actor or role.
Stabilized DIR may alter praxis conditions summarized by DIST	Orientation can affect later orientation–enactment separation without turning DIST into a core component.
Binding alters later orientation	Praxis can change the conditions under which subsequent claims and corrections are possible.
Praxis may change later orientation conditions	The model is recursive rather than a one-way analysis pipeline.

External inputs remain possible at every stage, including new difference, other actors, institutional change, material events, new evidence, and previously unknown conditions. These inputs prevent VECTOR from being read as a closed dynamical system. Detailed option-space, second-order agency, timing, and consequence relations are specified in Package III.

7. Result types, claim discipline, and external warrant

VECTOR terminates a bounded test by typing the result rather than producing a universal confidence score. The result set is deliberately non-hierarchical: a correct Suspend or Non-Capture may be more adequate than an inflated Robust Orientation. The model therefore separates the quality of a bounded result from any assumption that every analysis should converge on the strongest positive label.

7.1 Canonical result set

Result	Meaning	Critical boundary
Robust Orientation	The claim-relevant core survives materially relevant current pressure without material revision, within declared scope.	Always robust within scope; never robust simpliciter.
Revised Robust Orientation	The baseline materially failed, a revised claim was produced, and that revised claim subsequently survived relevant pressure.	Correction receives credit without rewriting baseline failure as survival.
Bounded Orientation	Sufficient directional support exists for the declared question without earning stronger robustness status.	Useful orientation may remain explicitly narrower than robustness.
Revision Required	The current claim fails, but a materially altered claim may be testable.	An intended repair is not yet a completed robust revision.
Multidirectional / Trade-Off	Several relevant directional relations survive, but no legitimate non-compensatory resultant is warranted.	Structured conflict is preserved rather than collapsed into a forced rank.
Incomparable	A meaningful comparison space exists, but no warranted dominance relation has been earned.	Incomparability does not imply equality.
Suspend	The question remains legitimate, but current warrant is insufficient.	Re-entry remains possible when evidence, timing, capacity, calibration, or rival pressure changes.
Non-Capture	VECTOR should not produce the requested directional relation in the current form.	Positive discriminative outcome; not a weaker version of Suspend.

The architecture excludes result labels such as `AbsoluteTruth` , `CertifiedCorrect` , and `FinalPosition` . Their absence is a model boundary, not a stylistic preference.

7.2 Robust Orientation is scoped pressure survival

Robust Orientation means that the claim-relevant core has survived the materially relevant current pressure at the declared scope. It does not mean that all imaginable tests have been exhausted, all external premises are established, or the result is final. Residual uncertainty may remain where it is not claim-critical for the current bounded question.

Robustness boundary
Robustness is pressure survival within declared scope. It is not universal superiority, finality, authority, moral correctness, or external premise warrant. A larger scope creates a larger burden: `Scope↑` ⇒ `Burden↑` .

7.3 Revision is not survival

The model explicitly separates a claim that survives from a claim that changes enough to become a new object of testing. If the baseline materially fails and a new formulation is introduced, the result cannot be counted as original-claim survival merely because the revised statement later works.

Claim continuity firewall

Revision $\neq$ Survival. There is no automatic claim continuity across material coordinate change. An exploratory probe that changes the frame or question does not retroactively warrant the original-frame claim.

This rule applies both to substantive wording and to material changes in the reference coordinates. Where θ , F , Q , π , or t changes in a claim-material way, continuity must be argued rather than presumed. A successful new claim may later earn Revised Robust Orientation; it does not erase the earlier reduction.

7.4 Suspend and Non-Capture are different results

Suspend means that the question is legitimate but present support is not sufficient. **Non-Capture** means that VECTOR should not produce the requested directional relation in the current form. The distinction matters because more evidence may resolve a suspension, whereas a Non-Capture result may require a different question, a different framework, or an explicitly external treatment rather than additional VECTOR processing.

Non-Capture may arise where reference continuity fails, the question is badly typed, the comparison would create an excluded person-worth ranking, the relevant structure cannot responsibly be compressed, or the necessary claim conditions cannot be preserved. It is therefore a positive boundary result: the framework discriminates where it should stop.

7.5 Warrant source boundary

Warrant conditions a DIR claim but is not a VECTOR core component. A material directional claim should make visible what carries its support. Depending on the bounded domain, sources may include PMS corpus structure, supplied empirical evidence, domain expertise, law or policy, historical material, explicit normative premises, or an externally calibrated decision model.

The inventory is not a universal warrant algorithm. Its purpose is to prevent VECTOR's own formal activity from becoming an epistemic source by default. A claim can be structurally well-typed and pressure-tested while remaining externally under-warranted.

External warrant boundary

VECTOR robustness $\neq$ external warrant. VECTOR may expose external dependencies, test how a claim changes if a premise fails, check typing and inference boundaries, and preserve an under-warranted status. It cannot manufacture historical evidence, medical knowledge, current law, empirical measurements, causal evidence, or role authority.

7.6 Admissible joint status

The repository therefore permits a joint status in which a claim is structurally robust under the declared VECTOR pressure while still externally under-warranted. This is not a contradiction. The two labels answer different questions: one concerns the behavior of the bounded claim under declared pressure; the other concerns whether the external premises needed to use that claim have actually been established.

The same separation applies to calibration. An externally supplied threshold or weighted decision model may be represented as the warrant/calibration source for a bounded DIR claim, but notation reuse does not convert that external calibration into VECTOR-generated warrant. If calibration fails, the appropriate response is claim reduction or Suspend rather than silently changing the threshold until the preferred direction reappears.

7.7 Package II integration contract

Sections 5–7 establish the technical core needed by all later repository artifacts. DIR provides bounded directional discrimination; ROT provides admissible claim-threatening perspectival pressure; RP records that pressure as audit state; DIST remains only a derived supporting description of source-carried orientation–enactment separation; reversibility remains a distinct path-sensitive dependency relation; and the final result must be typed without converting robustness into truth, authority, or external warrant.

Package II stop condition
At this point the reader should be able to distinguish core from state, operation from record, direction from total order, rotation from ordinary reframing, DIST from both core status and reversibility, and bounded robustness from external warrant. Package III can therefore add option space, second-order agency, consequence and cost topology, timing, convergence, uptake, structural attributability, typed ought relations, constraints, and corrigible praxis without redefining the core architecture.

8. Option space and transformable possibility

VECTOR does not treat possibility as a flat inventory of alternatives. A path can be conceivable without being visible, visible without being structurally available, structurally available without being effectively actionable for the relevant actor or role, and effectively actionable without satisfying the viability relation declared for the current question. The option-space architecture therefore exists to type possibility before later claims about agency, consequence, attributability, or practical necessity are made.

Canonical option-space notation
 $\mathcal{O}_{\{a, t\}}(O \mid F, Q)$ denotes the option space for actor or role a at time t , relative to object O , frame F , and question Q . The notation is supporting praxeological architecture, not an additional VECTOR core component.

The option space is actor- or role-relative because nominally identical environments can produce different effective possibility fields. Formal authority, access, information, means, dependencies, cost exposure, procedural permission, and temporal windows can differ by role. System-level possibility therefore cannot be silently converted into person-level capacity.

8.1 Option status is typed, not automatically ordered

The repository distinguishes five option statuses. Their sequence expresses increasing practical burden, but it does not authorize an automatic inference ladder. Every transition remains separately support-sensitive.

Option status	Meaning	What does not follow automatically
conceivable	The weakest status: the option can be represented or imagined.	Visibility, structural existence, actionability, or viability.
visible	The option is epistemically available to the relevant view.	That the option exists in the relevant structure or can be enacted.
structurally_available	The route exists in the declared field under the relevant structural relations.	That the relevant actor or role possesses practical reach to use it.
effectively_actionable	The actor or distributed actor configuration has effective practical reach under current conditions.	That the route satisfies the declared viability relation or is normatively preferred.
structurally_viable	The option can satisfy the viability relation declared for the current bounded question.	Universal superiority, moral rightness, or an obligation to select it.

Non-equivalence chain
`conceivable != visible; visible != structurally_available; structurally_available != effectively_actionable; effectively_actionable != structurally_viable`. The repository flag

`no_automatic_implication_ladder: true` applies throughout.

8.2 Option status and evidence status are different dimensions

A claim about what kind of option a route is must be kept separate from the evidential status of the claim itself. Missing evidence is not equivalent to either practical availability or practical unavailability. The model therefore permits unresolved option claims to remain unknown rather than forcing them into a binary availability state.

Unknown-status boundary

Unknown != Available and Unknown != Unavailable . An unresolved actionability claim may remain unresolved; analyst imagination does not fill the gap, and missing evidence does not prove absence.

Where actionability is material, evidence targets may include authority, access, means, information, an open time or decision window, dependencies on other actors or systems, practical control, relevant cost, and procedural permission. These fields are not a mechanical checklist whose completion proves actionability. They identify the relations that must be supported at the level required by the current claim.

8.3 Option invisibility and option-field misrepresentation

VECTOR separates a legibility condition from two opposite option-field status errors. **Option Invisibility** occurs where a relevantly real or realistically transformable option is absent from the current legibility field. It is **not by itself a failure mode**. It becomes an **Option Blindness** case only when that absence is converted into an unsupported exclusion, downgrade, or claim of non-existence, non-availability, or genuine lack of alternatives.

Type	Repository meaning	Boundary
Option Invisibility	A relevantly real or realistically transformable option is absent from the current legibility field.	Legibility condition; not by itself Option-Field Misrepresentation.
Option Blindness	Unsupported exclusion from the relevant current or realistically transformable option field.	Negative direction of Option-Field Misrepresentation.
Option Illusion	Unsupported inclusion in the relevant effective option field.	Positive direction of Option-Field Misrepresentation.

Option-field misrepresentation

OptionFieldMisrepresentation = UnsupportedExclusion OR UnsupportedInclusion . The assignment is evidence-bound: it does not presuppose access to a complete or final option field. Unknown must remain Unknown where the evidence does not warrant either direction.

The YAML retains several option-illusion families: frame illusion, capacity illusion, temporal illusion, role-transfer illusion, relational illusion, and cost-blind illusion. These are recurrent positive-misrepresentation families rather than peers of Option Invisibility. The list is diagnostic support rather than a closed ontology. A Special-Case Probe remains distinct from Option Illusion until its constructed possibility is imported into the effective option field without sufficient support. Likewise, a bounded negative after adequate option and transformation pressure is not Option Blindness merely because it excludes a route.

8.4 Branch status records what happened to a route

Option status concerns the practical standing of a route; branch status records how a route relates to subsequent praxis. The supported branch statuses are realized , rejected , blocked , deferred , and lost . Deferral is especially time-sensitive: an unselected option can remain available, narrow, become more costly, or disappear while no explicit rejection occurs.

Non-selection is not preservation

`non-selection` does not imply that the option remains available. The temporal option field may shrink or expand, and later availability must be reconstructed rather than assumed.

8.5 Option-field state and action corridors

Where a more explicit representation is useful, the option field may be recorded as $\mathcal{O}_t = (N_t, R_{\{0, t\}})$, where N_t represents option nodes and $R_{\{0, t\}}$ represents relations among them. This is a supporting formal target for changes in the possibility field; it is not a claim that the option field is complete, globally enumerable, or reducible to a decision tree.

Action corridors summarize bounded future reachability opened, preserved, narrowed, or closed by current conditions and praxis. They are useful precisely where a complete decision tree would overstate knowledge. A corridor can remain partially specified while still showing that a present action changes what later action remains possible.

Temporal option-field rule	Contract
Field may change	Option nodes and relations may be created, removed, opened, blocked, preserved, or restructured over time.
No monotonic shrinkage	Time can close routes, but it can also open routes or alter their relations. No universal one-way law is assumed.
No preservation by omission	Failing to select an option does not establish that the option remains available later.
No complete tree implied	A bounded action corridor is not a claim that all future branches are known.

8.6 Special-case probes can improve orientation without creating options

Counterfactual or special-case construction can be analytically useful even where the constructed route is not actually available. A probe may reveal which relation blocks the route, which authority distribution matters, which window closed, or which dependency would have to change. The analytical gain does not convert the probe into actor capacity.

Probe boundary

`valid probe != valid option`. Counterfactual discrimination may improve orientation while the relevant route remains structurally unavailable or non-actionable.

8.7 Section integration contract

The option-space architecture is therefore a typing layer between bounded orientation and praxis claims. It can preserve unknowns, distinguish hidden routes from fictional ones, represent role-relative actionability, and expose temporal changes in possibility without adding a new VECTOR operator. Later agency and attributability claims must consume these typed relations rather than infer capacity from mere conceivability or system-level possibility.

9. Second-order and distributed agency

Agency in VECTOR is not restricted to selecting among currently effective options. Praxis can sometimes alter the option field itself: opening a blocked route, preserving a window, creating a procedure, changing a sequence, recruiting another actor, or changing relations among existing options. The model calls this second-order agency, but only where a traceable transformation mechanism and effective capacity are supported.

9.1 First-order and second-order agency are distinct

Agency form	Repository meaning
First-order agency	Selection among options that are already effectively available in the relevant field.
Second-order agency	Effective change to the option field or to relations among options.

The distinction does not mean that first-order action lacks second-order effects. Selecting one branch can create dependency, close another route, raise later reversal cost, or allow a window to expire. The stricter second-order-agency claim concerns cases in which the relevant actor or distributed actor configuration can effectively alter the possibility structure itself in a way material to the current question.

9.2 Supported transformation families

The YAML permits several recurring transformation descriptions. They are not independent operators; they are ways in which a supported option-field change may be recorded.

Transformation	Illustrative structural effect
Create option	Add a materially new route to the current field.
Open blocked option	Change a relation that prevented an existing route from becoming actionable.
Preserve option	Maintain access, reversibility, information, or a temporal window that would otherwise be lost.
Close option	Remove or intentionally foreclose a route, possibly in order to stabilize another trajectory.
Change sequence	Reorder actions so that later options or costs change.
Change window	Extend, shorten, reopen, or otherwise alter the temporal conditions of actionability.
Change option relation	Alter dependency, exclusivity, precedence, access, or another relation among routes.

9.3 Mechanism burden

A material claim of second-order agency carries a mechanism burden. The burden exists because attractive counterfactuals are easy to imagine while effective transformations are often role-, capacity-, dependency-, and time-bound. Where the claim is material, the record should identify the actor or distributed configuration, the target change, the mechanism, the authority or effective capacity required, dependencies, the relevant window, supporting evidence, and the resulting node or relation change.

Formal target where useful

A supported option-field transformation may be represented as a change in `N` and/or `R_0`: ΔN and/or ΔR_0 . The notation records the target of the claim; it does not prove that the transformation mechanism exists.

Capacity firewall

`imagination_is_capacity: false` and `ability_to_imagine_is_effective_capacity: false`. A hypothetical rescue route can be a valid probe while remaining unsupported as effective second-order agency.

9.4 Distributed agency

Effective transformation can be distributed across roles. One role may possess authority, another information, another operational access, another a required resource, and another control over a procedural or temporal window. A transformation that exists at the configuration level therefore need not exist for any single participant acting alone.

The repository consequently permits `distributed_agency_possible: true` and rejects a sovereign-actor default. The analysis must preserve who controls which dependency rather than compressing collective capability into an imaginary individual capacity. This same boundary later prevents system-level carriage requirements from becoming person-specific authority claims.

Distributed-agency boundary

Collective or institutional transformability does not imply equal individual transformability. Where a route requires several roles, dependencies, permissions, or capacities, those relations remain part of the claim.

9.5 Second-order agency is not a fourth STRATA operation

The repository explicitly records `fourth_strata_operation: false`. Second-order agency is a VECTOR praxeological distinction about transformations in the option field. It does not add an operation to PMS-STRATA, redefine a STRATA transformation class, or establish an independent cross-repository operator by notation reuse.

9.6 Section integration contract

Second-order agency is therefore real but constrained. It expands analysis beyond fixed-option choice without making possibility infinitely plastic. Transformation claims survive only where the relevant actor configuration, mechanism, authority or effective capacity, dependencies, timing, evidence, and resulting field change remain traceable. Where those conditions are absent, the correct status may be hypothetical or unknown rather than effective agency.

10. Consequence, cost, timing, and convergence

Typed possibility and agency change what can responsibly be said about consequence. Action, non-action, delay, binding, reversal, externalization, and relation change can place effects on different bearers at different times. VECTOR therefore uses a consequence and cost topology rather than a single total-cost scalar, and it keeps timing and convergence as additional supporting test structures rather than hidden aggregation rules.

10.1 Consequence and cost topology

The repository recognizes several recurring cost relations. The list is a typing aid, not permission to infer a cost merely because a category exists.

Cost type	What the field is intended to preserve
Direct action cost	Burden produced by selecting or enacting the branch itself.
Non-action cost	Consequence structurally connected to refusal, omission, or non-selection.
Opportunity loss	Loss of a route, benefit, or later choice due to present allocation or commitment.
Window cost	Cost created by temporal closure, delay, or the need to preserve a decision window.
Binding cost	Burden created by commitment, lock-in, coordination, or other binding relations.
Reversal cost	Burden of correcting or reversing a path after enactment.
Externalized cost	Effect shifted to another bearer, role, population, system, or future time.
Relation-change cost	Burden arising because the action changes dependencies, access, trust, authority relations, or future possibilities.
Delayed / downstream cost	Later effect whose mechanism or dependency relation remains material to the current claim.

10.2 Consequence and cost evidence status

The consequence taxonomy must not create hidden costs by vocabulary alone. Material cost claims therefore carry a support status. These support statuses are distinct from provenance labels and from option statuses.

Evidence status	Meaning
observed_or_directly_established	The consequence is directly observed or otherwise directly established for the present claim.
externally_supported	An empirical, technical, legal, historical, causal, or other external source materially supports the relation.
traceably_inferred	The relation is inferred through a sufficiently explicit bearer, mechanism, temporal path, or dependency bridge.
hypothetical	The cost is being used as a probe or unresolved possibility rather than asserted as established.
unknown_or_residual	The analysis preserves unresolved consequence rather than inventing or deleting it.

Anti-inflation rule

A hidden or downstream cost may not be invented merely to save a preferred claim. Practical effect does not validate the orientation that caused it, and unsupported downstream cost remains residue rather than silently becoming evidence.

10.3 Costs are non-fungible by default

The repository sets `default_fungibility: false`. Costs borne by different roles, at different times, under different rights, dependencies, reversibility conditions, or exposure structures cannot be

automatically placed on one common scale. A model consumer may use an external commensuration or decision model where independently warranted, but that calibration remains external.

Cost claims are also role-bound and time-bound. A low reversal cost for one role can coexist with high irreversible exposure for another; a cost negligible at scene time can become decisive after a window closes; an apparent non-action route can transfer load rather than remove it. VECTOR keeps these relations distributed until the bounded question warrants a narrower reconstruction.

10.4 Timing

Timing is a supporting praxeological relation, not a universal optimization variable. A correct direction does not imply immediate enactment; delay is not automatically restraint; premature enactment is not equivalent to responsibility; and refusing premature action does not imply indefinite postponement. The timing question remains bound to the declared role, effective capacity, consequence topology, option field, reversibility, asymmetry, and relevant window.

Timing principle	Boundary
Direction vs enactment	A supported directional relation does not by itself determine when or whether enactment should occur.
Delay vs restraint	Delay may preserve a window, waste a window, shift cost, or merely defer a decision; its status is reconstructive.
Temporal transformation	Timing can change option nodes, option relations, reversibility, and consequence distribution.
Public/private difference	Critique or intervention can carry different timing and exposure relations depending on mode and audience.
No universal optimum	The model contains no globally optimal timing rule.

10.5 Convergence is not recurrence

VECTOR distinguishes repeated wording or structure from genuinely independent directional support. Lexical recurrence, structural recurrence, directional convergence, and robust convergence are different claims. Repetition inside one framework, one prompt ecology, one model family, or one inherited assumption can create apparent agreement without independent support.

Convergence level	Repository meaning
Lexical recurrence	The same or similar wording recurs.
Structural recurrence	A similar structural relation recurs.
Directional convergence	Distinct local DIR (Direction) relations support compatible bounded direction.
Robust convergence	The direction survives material provenance, ROT (Rotation), rival, and non-compensation pressure.

Convergence firewall

recurrence != convergence ; framework habituation != independent support ; AI convergence != adequacy .

10.6 Robust convergence is conjunctive and non-compensatory

A stronger convergence claim requires separately warranted and distinctly constrained local directional relations, provenance and echo control, relevant ROT (Rotation) pressure, preserved difference, rival sensitivity, counterfactual sensitivity, non-compensation, absence of critical regression, and a real possibility of collapse or Non-Capture. These conditions form a conjunctive burden rather than an additive vote count.

Where a resultant orientation is warranted, it is a new bounded claim reconstructed from surviving relations. It must carry the local source DIR (Direction) relations, preserved differences, the common direction, and the claim bounds, frames, questions, and pressures that earned the result. The broader resultant cannot retroactively validate weak local inputs.

No algebraic resultant

A VECTOR resultant is not a vector sum, vote, average, weighted aggregate, or scalar score. No robust resultant is itself a mandatory valid outcome; the correct result may remain trade-off, incomparability, collapse, Suspend, or Non-Capture.

10.7 Section integration contract

Consequence, cost, timing, and convergence extend the bounded claim architecture without turning VECTOR into an optimizer. They preserve who bears which effect, with what support and at what time; they permit changing option relations without assuming monotonic progress; and they allow multiple local directions to be tested for meaningful convergence without replacing non-compensation with hidden aggregation.

11. Orientation uptake and structural attributability

A relation present in an analysis is not automatically part of an actor's praxis situation. VECTOR therefore inserts an explicit uptake dependency before claiming that its own orientation changed the structural conditions under which later action or non-action is attributable. This is a narrow transition rule. It does not rewrite the separate baseline attribution architecture supplied by PMS-LOGIC.

11.1 Uptake is a staged practical transition

The repository records five distinguishable states: presented, actor-accessible, understood, praxis-available, and enacted. The sequence is not a psychological theory and does not require agreement. It identifies where an orientation may fail to become practically usable even after it has been communicated.

Stage	Repository meaning
presented	The claim or orientation has been communicated or made available in some form.
actor_accessible	The actor can actually access the orientation in a usable form.
understood	The relevant claim and support burden are intelligible enough to contest, reject, or respond to.
praxis_available	The relation has entered the practical question-space sufficiently to be checked, rejected, accepted, refused, contested, or acted upon.
enacted	The orientation has entered actual praxis through action or another materially relevant enactment relation.

Canonical anti-abuse rule

`presented != actor_accessible != understood != praxis_available != enacted` . **Being told is not uptake.** Disagreement is not failed uptake by definition, and presentation alone does not create an orientation-mediated attribution increase.

11.2 Baseline PMS-LOGIC attributability remains external

PMS-LOGIC supplies a separate baseline relation of structural attributability. VECTOR must not insert an awareness or uptake requirement into that baseline. Where effective capacity and attributable consequence are otherwise structurally established under PMS-LOGIC, baseline attributability does not generally wait for conscious uptake of VECTOR's orientation.

Baseline firewall

`BaselineAttributability_LOGIC != ΔAttributability_orientation` . VECTOR uptake gates apply only to the additional claim that VECTOR-mediated orientation changed the actor's attribution conditions.

This distinction protects both directions of error. VECTOR cannot reduce an independently warranted PMS-LOGIC baseline merely because an actor lacked VECTOR uptake, and it cannot increase attribution merely because the analyst or another participant possessed a better reconstruction.

11.3 Orientation-mediated attribution change is dependency-bound

Where an orientation-mediated change in structural attributability is material, the repository expects a conjunction of dependencies rather than a rhetorical moment of notification. Relevant orientation, external premise support, actor uptake, effective capacity, a consequence relation, and role/time conditions must be sufficiently supported for the bounded claim.

Orientation-mediated attribution gate

`RelevantOrientation + ExternalPremiseSupport + ActorUptake + EffectiveCapacity + ConsequenceRelation + Role/TimeConditions` . This is a dependency structure, not an algorithm, score, moral inference, or universal threshold.

External premise support remains genuinely external where the claim depends on empirical, legal, technical, causal, historical, capacity, or other domain facts that VECTOR cannot establish. Structural robustness cannot substitute for those premises. Effective capacity may be direct or may depend on a supported second-order-agency mechanism; analyst imagination is insufficient.

11.4 Scene time and analysis time remain distinct

A later analyst can know more than the actor knew at scene time. That later knowledge does not retroactively create earlier actor legibility, understanding, authority, access, capacity, or an open window. Where the distinction is material, records should keep scene time `t_s` separate from analysis time `t_a` .

Temporal attribution firewall

`Legibility_analyst(t_a)` does_not_imply `Legibility_actor(t_s)` and `knowledge_at_t_a` does_not_imply `effective_capacity_at_t_s` .

11.5 Structural attributability is non-monotonic

Better orientation does not operate as an accusation ratchet. It can increase structural attributability, but it can also decrease it, retype it, or leave the support insufficient. A better reconstruction may reveal that an apparent option was not actionable, that authority belonged to another role, that the temporal window had already closed, or that an external premise was unsupported.

Possible effect of improved orientation	Interpretation
Increase	A materially relevant relation becomes sufficiently actor-accessible, capacity-relevant, and consequence-linked.
Decrease	A presumed capacity, option, authority, or consequence relation is shown not to hold.
Retype	The relevant attribution relation belongs to a different role, time, mechanism, or practical category.
Remain insufficient	One or more dependencies remain unresolved or under-supported.

11.6 Technical terminology and human boundary

The technical primary term is **structural attributability**. The phrase **loss of structural innocence** is retained only as secondary shorthand for a narrower technical condition and is not an independent claim type. The terminology must not be allowed to import ordinary-language guilt or moral accusation.

Human boundary

Increased structural attributability does not imply increased authority. Loss of structural innocence is not guilt, blame, liability, or moral fault. VECTOR can type a practical attribution relation without ranking the person who occupies it.

11.7 Section integration contract

Uptake and structural attributability therefore connect orientation to praxis without making communication coercive, awareness a universal prerequisite for PMS-LOGIC, or improved orientation automatically accusatory. The transition is actor-, role-, time-, capacity-, consequence-, and evidence-sensitive, and its dependencies remain independently defeasible.

12. Ought relations, constraints, and corrigible praxis

VECTOR uses typed Ought relations because ordinary must-like language can collapse practical necessity, accountability, prescription, institutional requirement, and moral command into one category. The model does not introduce a moral operator family. Instead it distinguishes what a bounded orientation can support, what requires external normative premises, and what can follow from an operative constraint without conferring legitimacy on the constraint or its current bearer.

12.1 Ought typology

Type	Meaning	VECTOR-alone status
<code>OUGHT_choice</code>	Outcome-directed prescriptive choice claim.	Boundary type; not generated by VECTOR alone.
<code>OUGHT_account</code>	A relevant relation <i>R</i> must remain accounted for in subsequent praxis.	Positive VECTOR relation when its orientation-mediated dependencies carry.
<code>OUGHT_constraint</code>	Constraint-relative practical necessity identifying a response class required to maintain operative condition <i>G</i> .	Positive VECTOR relation when the constraint and effective option dependencies carry.
<code>OUGHT_moral</code>	Categorical moral claim.	Boundary type beyond VECTOR-alone warrant.

Moral conversion boundary

VECTOR Ought notation does not import moral legitimacy, legal duty, obedience, or person rank. A must-like structural relation may be real while its source, legitimacy, fairness, or moral standing remains separately contested.

12.2 `OUGHT_account` : a relation must remain accounted for

`OUGHT_account` is narrower than a command to choose. Its canonical meaning is that a sufficiently warranted and praxis-available relevant relation *R* may no longer be erased from subsequent praxis without claim loss. The actor may still reject the analyst's conclusion, challenge the frame, refuse action, select a costly branch, revise the goal, or accept a tragic result.

Canonical `OUGHT_account` relation

`OUGHT_account(R) = R must remain accounted for in subsequent praxis.`

The technical primary effect is a possible orientation-mediated change in structural attributability. The relation is calibration-sensitive and supplies no universal threshold. It neither replaces baseline PMS-LOGIC attributability nor converts the presence of a relation into guilt, blame, liability, moral fault, or a presumption that the actor must obey VECTOR.

12.3 `OUGHT_constraint` : constraint-relative practical necessity

`OUGHT_constraint` identifies a response class required to maintain an operative condition *G* under a correctly typed effective option field. Its force is conditional: if *G* remains operative and the effective response field is correctly reconstructed, some response may be practically required for *G* even where no morally good option is available.

A valid constraint record should preserve at least the operative condition, what maintains and may modify it, claimed authority, burden bearers, exit relations, alternative carriers, transformation routes, consequences of abandonment or revision, unresolved relations, and any external legitimacy question. This structure prevents the fact of constraint from laundering its source or legitimacy.

UGHT_constraint dependency	Requirement
Operative G	The maintained condition is actually part of the bounded practical question.
External premise support	Material factual, legal, technical, institutional, or causal premises are supported outside VECTOR where required.
Effective option field	Candidate responses are typed by effective rather than imaginary availability.
Non-empty response class R_G	At least one effective response remains that satisfies the relevant G-condition.
Constraint relations visible	The relations that make the response class necessary remain inspectable.
Member satisfaction	Each member of R_G satisfies the relevant G-condition.

Constraint-source boundary

$\text{Source}(G) \neq \text{Legitimacy}(G)$. A constraint can be structurally operative while its source remains unjustified, contested, revisable, or illegitimate. If G is materially revised, the model requires a new constraint claim: $\text{Revision}(G) \Rightarrow \text{new constraint claim}$.

12.4 System carriage does not identify a person who must carry

A system can require continuity, output, stabilization, or another function without thereby identifying which person must bear the load. The model therefore separates the existence of a carriage requirement from the assignment, legitimacy, or authority of a particular bearer.

System/person firewall

$\text{SystemCarriageRequired} \neq \text{PersonXMustCarry}$. Assignment of a carrier requires additional role, capacity, authority, legitimacy, distribution, and consequence claims.

12.5 Fixpoint Requirement is functional, not authoritative

In some conflict or continuity configurations, a stabilizing function is required even where integrative resolution is unavailable. The fixpoint may be a role, procedure, institutional mechanism, escalation path, temporary coordination function, or another stabilizing structure. It is not necessarily a person and does not create leadership virtue or normative supremacy.

Fixpoint firewall

$\text{FixpointRequirement} = \text{functional requirement}$; $\text{FixpointRequirement} \neq \text{BearerAuthority}$; requirement of a fixpoint does not imply legitimacy of a particular fixpoint.

The functional requirement can therefore be real while the current bearer remains challengeable. Selection, authority, limits, burden distribution, reversibility, dignity constraints, and legitimacy each require their own support. Restoring continuity does not by itself show that the arrangement is normatively acceptable.

12.6 Tragic Constraint is evidentially bounded

A Tragic Constraint is a bounded configuration of `UGHT_constraint` in which every currently evidenced effective response in `R_G` carries material residual loss and no currently evidenced effective non-lossful route removes that structure within the declared window, capacity field, and dependency structure. The claim is about the present evidenced field, not metaphysical impossibility.

Tragic Constraint burden	Required relation
Operative constraint	A valid operative G relation remains in force for the current bounded claim.

Tragic Constraint burden	Required relation
Effective field	The option field has been typed against actual capacity rather than hypothetical rescue routes.
Residual loss	Every member of <code>R_G</code> carries relevant loss.
Transformation audit	Second-order mechanisms, authority/capacity, dependencies, and the relevant window have been seriously checked.
No evidenced non-lossful route	No currently evidenced effective route or transformation removes the residual-loss structure.

Tragic Constraint falsifier

The classification is falsified by an evidenced effective non-lossful route, including an evidenced second-order transformation that opens such a route within the declared window, capacity field, and dependency structure. Mere imagination of a rescue route is not enough.

Tragic Constraint does not legitimate the existing system, prove that losses are just, identify who deserves to carry them, or convert harm into necessity because no alternative was imagined. Conversely, the existence of residual loss does not eliminate practical necessity: there can be cases where no good option survives while some response remains required for an operative `G`.

12.7 Corrigible consequence-carrying praxis

The model uses **corrigible consequence-carrying praxis** for a stance in which orientation remains revisable while the actor or distributed configuration does not erase sufficiently warranted consequences, constraints, capacities, and residue that have become praxis-available. The earlier label **Adult Orientation** is retained only as supporting shorthand for this already typed combination.

Carried relation	Specification role
Bounded relevant orientation	The stance remains tied to the claim that actually survived the current burden.
Preserved corrigibility	New evidence, rival pressure, coordinate change, or claim regression can revise the orientation.
Relevant consequence carrying	Material consequence relations that have become sufficiently warranted are not erased from praxis.
Capacity limits	The stance does not imply sovereign control or unlimited effective agency.
Residue / Λ	Unresolved loss, uncertainty, or non-integrated relation can remain visible.
Orientation-enactment source conditions	Relevant separation is carried through source relations without reifying derived <code>DIST</code> .
Distributed agency where relevant	Praxis can depend on multi-role capacity rather than a sovereign individual.
No authority conversion	Orientation does not confer authority over others.

Adult Orientation reduction firewall

```
AdultOrientation != AdultReciprocity_EDEN; AdultOrientation != MIP maturity;
AdultOrientation != AdultPerson; AdultOrientation != person rank; successful outcome !=
evidence of the stance. The shorthand has no independent theoretical discrimination, no separate
dependency node, and no metric, threshold, operator, or result type.
```

MIP supplies a boundary precedent for keeping situational constructs separate from dignity or person rank; it does not supply the warrant for the Adult Orientation label or stance. The label remains disposable without loss of the lower-level discrimination that carries its content.

12.8 Package III integration contract

Sections 8–12 extend VECTOR from bounded orientation into a typed praxis architecture without turning it into a universal action engine. Option status remains distinct from evidence status; second-order

agency requires an effective mechanism; consequence and cost relations remain role-, time-, and evidence-sensitive; uptake gates only VECTOR-mediated attribution change; practical Ought relations remain non-moral by default; system necessity does not manufacture person authority; Tragic Constraint remains falsifiable; and Adult Orientation remains supporting shorthand only.

Package III stop condition

At this point the reader should be able to reconstruct how a bounded VECTOR orientation can enter a praxis field without silently creating options, capacities, costs, uptake, blame, authority, legitimacy, or moral command. Package IV can therefore specify test burden, failure and reduction, rival superiority, non-capture, repository interfaces, provenance, computational representation, and Reader boundaries without reopening the praxis architecture.

13. Test burden, failure families, and falsifiability

VECTOR treats pressure, loss, and reduction as part of the model contract rather than as optional criticism applied from outside. A bounded orientation claim is therefore not considered adequately tested merely because it can be restated across several favorable descriptions. The repository representation requires an explicit **bounded test burden**: a finite but revisable set of materially distinct pressures sufficient for the current claim, scope, and calibration. Closing that burden is a stop discipline. It is not truth closure, finality, or immunity from later rival pressure.

Bounded closure, not truth closure

`TestBurdenClosed(current claim)` means only that known claim-critical vulnerabilities have received materially distinct pressure sufficient for the declared present scope. It does not mean that all possible tests have been exhausted, that future evidence cannot alter the result, or that VECTOR has validated itself.

13.1 Minimum burden for a materially load-bearing VECTOR claim

The YAML block `bounded_test_burden_closure` records the minimum burdens that must be visible where they are material to the claim. Their role is to prevent a nominally complete record from hiding the very relation under pressure.

Burden	Repository requirement	What failure risks
Coordinates and baseline	Reference, frame, question, perspective, temporal position, and baseline claim are declared.	Untyped comparison or retrospective coordinate drift.
Relevant Ω / Θ conditions	Constraint and temporal conditions are visible or explicitly left as residue.	False option, false reversibility, or invented necessity.
Current DIR	The directional result actually being pressured is stated.	Rotation without a stable claim target.
Material ROT families	Relevant perspective changes are applied rather than selectively omitted.	False invariance or rotation laundering.
Orientation-enactment source conditions	Source relations behind derived DIST are tested.	Reification of a derived descriptor.
Option field	Options are mapped only to the level needed for the claim.	Naturalized option field or unbounded option fantasy.
Option-field discrimination	Invisibility/discovery, unsupported exclusion, and unsupported inclusion are pressured separately.	Premature contraction, unsupported necessity, or inflated possibility field.
Transformability	Real second-order option-space transformation is tested where material.	Fatalism or voluntarism.
Consequence topology	Relevant direct, indirect, delayed, externalized, opportunity, transformation, and reversal costs are typed.	Local-benefit collapse or hidden-cost invention.
Serious rival	At least one rival has real winning conditions where rivalry is material.	Self-confirming test design.
Counter-scene / counter-position	A materially relevant alternative scene or position can pressure the result.	Perspective privilege disguised as robustness.
Provenance / dependency	Load-bearing origin and vulnerability relations are auditable.	Borrowed warrant, hidden dependency, or lineage confusion.
Known untested questions	No known claim-critical untested question is hidden at closure.	Premature closure.
Λ / residue	Relevant unresolved loss or uncertainty remains visible.	False completeness.

These are not universal checkboxes. Their requiredness is claim-strength- and claim-type-dependent. A narrow bounded comparison may not require every downstream praxis field, while a claim about option transformation, structural attributability, or Tragic Constraint carries additional burdens. The governing relation is therefore not `more fields = more truth`, but `stronger claim = stronger visible burden`.

13.2 Failure families as architecture attack surface

Chapter 18 of the paper organizes predictable misuse into eight failure families. These are not merely examples of careless application. They identify transitions the architecture itself makes tempting: a legitimate move becomes a drift point and then a stronger claim type is smuggled in without new warrant.

Failure family	Typical drift	Boundary to preserve
<code>direction_and_order_drift</code>	Local direction becomes score, total order, universal rank, or authority.	Direction does not manufacture metric rank or authority.
<code>rotation_and_perspective_drift</code>	Perspective change becomes frame substitution, desired-result search, or endless postponement.	Perspectival freedom is not freedom to alter the test invisibly.
<code>dist_and_reversibility_drift</code>	Derived separation becomes a virtue score or reopening becomes historical reset.	DIST is derived; reversibility is a budget, not a virtue score.
<code>possibility_and_agency_drift</code>	Conceivable becomes available/capable, or current constraints become fatalistically fixed.	Constraint-bound agency is neither fatalism nor possibilism.
<code>consequence_and_timing_drift</code>	Unseen consequence is invented, heterogeneous cost is scalarized, or delay is treated as neutral.	Unseen is neither zero nor invented; time can change the field.
<code>convergence_and_self_test_drift</code>	Recurrence or internal survival becomes truth confirmation.	Convergence and self-test remain pressure devices, not self-validation.
<code>ought_constraint_and_stance_label_drift</code>	Capacity, necessity, or supporting stance language becomes moral duty, legitimacy, or person rank.	Practical relations do not generate moral or authority status automatically.
<code>meta_authority_and_self_immunization_drift</code>	VECTOR becomes tribunal, absorbs rivals, or treats criticism as merely another VECTOR input.	A critique of VECTOR may remain external to VECTOR.

Meta-authority firewall

The capacity to describe a failure mode does not immunize the framework from that failure. Naming rival domestication does not prove that a particular rival remained independent; naming self-reduction does not prove that self-reduction occurred. The relevant evidence must remain external where the claim depends on external evidence.

13.3 Falsifiability and reduction tests

The machine-readable block `falsifiability_and_reduction` maps explicit failure tests to possible consequences. The governing rule is: **self-application must preserve the possibility of self-reduction**. VECTOR self-application is therefore not final validation.

Failure test	Potential consequence	Current note
Standalone redundancy	Standalone scope or status reduction	Still live; current standalone status is already scope-reduced.

Failure test	Potential consequence	Current note
DIR discrimination failure	DIR reduction or scope reduction	Current scope narrowed by same-burden rival pressure.
ROT discrimination failure	ROT reduction or scope reduction	Current scope narrowed by same-burden rival pressure.
DIST redundancy	Component reduction	Triggered: DIST removed from VECTOR core.
Option-space discrimination failure	Simplify, narrow, recalibrate, or remove the supporting distinction that failed to discriminate	Failure if Invisibility/Blindness, bounded negative/Blindness, or Probe/Illusion cannot remain distinct, or if Blindness/Illusion become post hoc stabilization vocabulary.
Second-order agency failure	Supporting architecture reduction	Retained under mechanism burden.
Cost-topology failure	Supporting architecture reduction	Retained under non-fungibility and evidence limits.
Convergence failure	Robustness / closure claim reduction	Retained as narrowed test architecture.
Self-test failure	Architecture pressure	Internal inconsistency is pressure, not automatic proof of total failure.
Ought-typology failure	Typology reduction	Retained only with uptake, constraint, and moral-conversion firewalls.
Adult Orientation redundancy	Supporting shorthand only	Triggered: independent construct status removed.
Calibration failure	ClaimReduction or Suspend	No automatic rescue by reducing scope rhetorically.
Rival superiority	Domain deference, scope reduction, or standalone reduction	Triggered at scope level: E.23.
Failure of self-reduction	Theory self-immunization failure	Meta-level failure if losses are only named but never permitted.

13.4 Reduction ladder

Failure is deliberately not binary. VECTOR uses a reduction ladder so that local claim loss need not be inflated into total theory collapse, while architecture-level failure cannot be hidden behind local survival.

Level	Reduction form	Meaning
1	Claim reduction	The bounded claim weakens, regresses, or loses a relation.
2	Scope reduction	The construct survives only in a narrower class of questions or domains.
3	Component reduction	A candidate component is removed or demoted from the architecture.
4	Domain non-capture	VECTOR declines or loses primary standing for the relevant domain/burden.
5	Architecture reduction	A larger supporting or structural subsystem is reduced.
6	Standalone reduction	VECTOR no longer warrants the claimed independent theoretical status.

Negative result is not theory failure

Collapse, Trade-Off, Incomparability, Suspend, Revision, or Non-Capture may be correct results of a properly bounded VECTOR analysis. A theory of orientation fails not because every case lacks a directed answer, but when it cannot distinguish when direction is and is not warranted, or cannot permit its own constructs to lose status under real pressure.

13.5 The permissible conclusion: no robust VECTOR

The strongest stop condition remains live at every scale. A case may end with no robust VECTOR available for that case; a domain may fall outside VECTOR adequacy; an ecosystem comparison may yield no robust resultant; and the theory may fail to earn standalone status. These are not ceremonial escape clauses. They are admissible outcomes of serious calibration, rivalry, and self-reduction.

Strongest self-reduction condition

VECTOR's continued status is conditional on the discrimination its distinctions actually perform. If the architecture cannot preserve a real route to the conclusion that VECTOR itself is not adequate for the current burden, its self-pressure has become immunizing rather than falsifiable.

14. Rival superiority, reduction status, and non-capture

Rival pressure is load-bearing because VECTOR must not gain authority merely by translating competing analyses into its own vocabulary. A serious rival may remain non-PMS, may use a different calibration architecture, and must have real winning conditions. Where the rival carries the relevant bounded burden better, VECTOR is required to yield domain ownership, narrow scope, reduce a component, or reduce its standalone claim as appropriate.

14.1 Two forms of rival victory

The current repository distinguishes **specialist rival superiority** from **same-burden rival superiority**. This distinction is critical because only the latter directly pressures the remaining DIR-ROT core on VECTOR's own bounded orientation burden.

Rival form	Meaning	Canonical case	Current consequence
Specialist rival	A narrower technical burden is better handled by a specialist model VECTOR should not own as primary.	E . 21	Domain-task deference; not by itself a same-burden DIR/ROT scope reduction.
Same-burden rival	A non-PMS rival addresses the same bounded orientation/discrimination question and outperforms VECTOR on that burden.	E . 23	DIR scope, ROT scope, and Standalone VECTOR scope may be reduced.

E.21 is not E.23

E.21 demonstrates that a calibrated specialist model can win its native technical task. That is an important anti-capture result, but it does not by itself show that DIR or ROT fail on VECTOR's own bounded orientation burden. E.23 is stronger because the rival and VECTOR address the same bounded question.

14.2 Same-burden scope boundary

Appendix E.23 supplies the current strongest rival result. In the tested class, a calibrated non-PMS multi-criteria sensitivity architecture carries an externally calibrated, commensurable comparison more directly than VECTOR. VECTOR therefore **cedes primary directional and perspectival status** for that bounded burden unless DIR or ROT add discrimination not already supplied losslessly by the rival.

Re-expression of the rival's result inside VECTOR notation does not reverse this loss. The rival's external calibration remains the warrant source. VECTOR may record the externally calibrated relation as an input or bounded directional fact only while preserving that provenance.

Rival assimilation prohibited

Rival calibration translated into VECTOR notation is still rival calibration. notation reuse != endogenous warrant and translation != victory . A rival must be allowed to remain external enough to reduce VECTOR.

14.3 Current reduction status

The current reduction table is not hypothetical. It records losses already enacted in the final paper and must be preserved by downstream templates, cases, Reader views, and release metadata.

Construct	Current status	Reduction pressure / remaining burden
DIR (Direction)	retain_scope_narrowed	E.23: same-burden rival superiority in externally calibrated commensurable multi-criteria orientation.
ROT (Rotation)	retain_scope_narrowed	E.23: sensitivity architecture may already carry relevant priority variation.

Construct	Current status	Reduction pressure / remaining burden
DIST	derived_supporting_only	E.22 / Ch. 19.5: matched-case redundancy; removed from VECTOR core.
Option taxonomy	retain_evidence_typed_and_narrowed	Unknown boundaries remain active; Invisibility != Blindness; bounded negatives remain distinct from Blindness; Special-Case Probes remain distinct from Illusion; bidirectional assignments must remain corrigibly auditable and the taxonomy must reduce if those distinctions fail.
Second-Order Agency	retain_narrow	Actor/configuration, mechanism, capacity/authority, dependencies, window, and evidence must be traceable.
Cost topology	retain_narrow	Non-fungible discrimination retained; unsupported hidden costs remain Hypothetical/Unknown.
Convergence	retain_as_test_architecture_narrowed	Echo control required; recurrence cannot generate resultant orientation by itself.
Bounded Test-Burden Closure	retain_calibration_sensitive_narrowed	Threshold basis remains visible; closure is not finality.
Ought typology	retain_with_firewall_narrowed	Uptake gate, constraint audit, and external moral conversion remain required.
Adult Orientation	supporting_shorthand_only	Deletion/discrimination test; no independent construct status remains.
Standalone VECTOR	provisional_retain_scope_reduced	E.23 same-burden rival superiority; E.21 remains related specialist-domain deference only.

Current core after enacted reduction

{DIR, ROT, DIST} → {DIR, ROT}
 Adult Orientation → supporting shorthand
 Standalone VECTOR → provisional retain, scope-reduced

14.4 Positive boundary after reduction

Reduction does not leave an empty architecture. What remains is narrower and more explicit. The retained positive boundary includes bounded directional discrimination, perspective as a testable variable, reversibility-sensitive orientation, discrimination between real and illusory possibility, constraint-bound second-order agency, non-fungible consequence topology, bounded test-burden closure without finality, practical consequence without moral automation, corrigible consequence-carrying praxis, legitimate Non-Capture, and structural attributability without added authority.

These positive claims apply only where the remaining DIR (Direction)–ROT (Rotation) architecture adds discrimination not losslessly supplied by PMS structures, specialist methods, or same-burden rivals. Retention is therefore a bounded status, not a presumption of theoretical priority.

14.5 Non-Capture as positive boundary and result

Non-Capture must remain a legitimate outcome. It means that the requested claim cannot responsibly be produced within VECTOR under the declared conditions without violating type, reference, warrant, or residue boundaries. Non-Capture is not equivalent to Incomparability, Suspend, or total theory failure.

Possible Non-Capture trigger	Why VECTOR should stop
Reference continuity failure	The comparison no longer addresses the same relevant object/relation.
Badly typed question	The requested output exceeds the kinds of claim the architecture can warrant.

Possible Non-Capture trigger	Why VECTOR should stop
Category jump	A structural result is being converted into another claim family without warrant.
Person-worth ranking request	The human boundary prohibits conversion of VECTOR relations into dignity/person ranking.
Claim-critical residue not responsibly compressible	Producing one result would erase relevant Λ .
Required claim conditions cannot be preserved	The model cannot retain the coordinates or evidence needed for a valid bounded claim.

External critique may remain external

VECTOR does not require every successful criticism to become a VECTOR result. A rival, empirical finding, legal constraint, moral argument, or conceptual critique may remain external to VECTOR and still require reduction. Non-Capture and rival superiority protect that possibility from assimilation.

15. Records, cases, provenance, and dependency interfaces

The repository externalizes several audit layers that the paper introduces but should not force a reader to reconstruct manually on every use. These interfaces make claims, provenance, dependencies, and case pressure inspectable. They do not create warrant merely by being complete or machine-readable.

15.1 VECTOR Record interface

Appendix B provides the minimal theoretical record. The repository companion `model/VECTOR-Record.template.yaml` is intended to translate that record into a reusable scaffold. It is a reference structure, not a JSON-schema-style validation authority and not an automatic direction engine.

Canonical record field	Role
<code>reference</code>	Declares the bounded object/relation O.
<code>frame</code>	Declares the comparison frame F without confusing it with PMS $\square$.
<code>question</code>	Declares the bounded question Q.
<code>position</code>	Declares perspective π and temporal position t as needed.
<code>baseline_claim</code>	States the claim under pressure before current testing.
<code>dir</code>	Stores the current bounded Direction relation/result.
<code>calibration</code>	Records external/domain calibration and its source where material.
<code>rot_profile</code>	Stores RP, not the ROT operation family itself.
<code>dist_derived</code>	Stores derived orientation-enactment separation only where useful.
<code>reversibility</code>	Records revision/reversal conditions without treating them as DIST.
<code>option_space</code>	Types relevant option states and evidence states.
<code>transformability</code>	Records second-order mechanism evidence where claimed.
<code>consequence_topology</code>	Records relevant heterogeneous consequence/cost relations.
<code>constraint</code>	Records operative G, carriers, alternatives, burden, exit, and transformation relations where material.
<code>orientation_uptake</code>	Records actor-accessibility/understanding/praxis availability only when relevant.
<code>pressure</code>	Records applied ROT, rival, evidence, counter-scene, or other material pressure.
<code>result</code>	Records the bounded result, reduction, suspension, or non-capture outcome.

Field requiredness depends on claim type and claim strength. VECTOR therefore does not assign default numeric fields for a DIR score, rotation-count score, DIST percentage, responsibility score, adulthood score, or universal confidence percentage.

Record boundary

`record completeness != claim correctness and schema validity != claim warrant`. An incomplete record does not automatically make a claim false; a complete or syntactically valid record does not make a claim true.

15.2 Case suite interface

The paper-linked case suite is indexed by `cases/index.yaml` and contains twenty-three cases, E01–E23. The repository cases are pressure records tied to Appendix E. They test distinctions, collapse conditions, non-capture, rivalry, and enacted reductions. They are not a statistical sample and do not become stronger evidence merely by being numerous.

Load-bearing case	Repository role
E20	Demonstrates legitimate Non-Capture.
E21	Demonstrates specialist rival superiority and domain-task deference.
E22	Performs the DIST necessity test and records architecture/component reduction.
E23	Demonstrates same-burden rival superiority and current DIR/ROT/Standalone scope reduction.

Case-suite boundary

`case count != evidential weight and case suite != empirical validation`. Cases expose and test the architecture under declared conditions; they do not establish population-level truth or general empirical prevalence.

15.3 Claim provenance interface

Appendix F distinguishes claim origin/status using the labels `C`, `R`, `N`, and `P`. The repository companion `reference/Claim Provenance.yaml` is the intended machine-readable map for these relations.

Label	Meaning	Does not imply
<code>C</code>	Corpus-carried	Strongest or automatically warranted claim.
<code>R</code>	VECTOR reconstruction	Simple quotation or unchanged inheritance.
<code>N</code>	VECTOR-new	Superior or self-warranting claim.
<code>P</code>	Proposed / provisional formalization	Weakness by definition or epistemic inferiority to <code>C/R/N</code> .

The provenance categories encode origin and formal status, not epistemic rank. They imply no ordering, no automatic evidence strength, and no authority inheritance. Ecosystem lineage can make a claim traceable without making the upstream source responsible for VECTOR's new generalization.

Provenance boundary

`ProvenanceCompleteness != TheoreticalAdequacy`. Complete lineage can reveal where a claim came from and still leave the claim false, under-calibrated, redundant, or inferior to a rival.

15.4 Source lineage

`PMS.yaml` is the only hard Base dependency in the lineage block. Other PMS ecosystem sources provide provenance, pressure, precedent, or reconstructive input; they are not all required runtime dependencies for every VECTOR application.

Lineage source	VECTOR relation
MIP	Trajectory, power, feedback, and dignity-boundary precedent.
PMS-ANTICIPATION	Future orientation, timing, restraint, and effective capacity.
PMS-CRITIQUE	Direction, correction, review distance, and follow-up.
PMS-CONFLICT	Shrinking options, carriage, fixpoint, and tragic non-integrability.
PMS-EDEN	Truthful coordination, learned legibility, counter-scenes, and reciprocity boundary.
PMS-LOGIC	Effective capacity and baseline structural attributability.
PMS-SEX	Exit realism and downstream cost.
PMS-DISCIPLINE	Bounded adequacy, correction before coherence, stop/refusal, and claim ceilings.
PMS UNDER LOAD	Pressure and self-reduction precedent; not warrant.
PMS-STRATA	Local superiority, non-compensation, claim reduction, rival superiority, and non-capture.

This list is a provenance map, not a hidden runtime import graph. In particular, MIP offers boundary precedent rather than warrant for Adult Orientation; UNDER LOAD supplies pressure rather than truth; STRATA supplies methodological precedent without making R0T identical to PROJECT_AS .

15.5 Dependency interface

reference/Dependency Map.yaml externalizes load-bearing dependency relations from Appendix F.9. The governing rule is: **dependency transmits vulnerability, not automatic warrant.**

Load-bearing node	Dependency role
RobustOrientation	Depends on typed DIR, relevant pressure, claim-loss routes, warrant, calibration, provenance, and rival conditions.
OUGHT_account	Depends on actor uptake, effective capacity, consequence relation, and role/scene-time conditions for orientation-mediated uplift.
OUGHT_constraint	Depends on operative G, externally supported premises, effective option field, and constraint relations.
TragicConstraint	Depends on valid constraint structure, residual loss, transformation audit, and no evidenced effective non-lossful route.
CorrigibleConsequenceCarryingPraxis	Depends on lower-level orientation, corrigibility, consequence, capacity-limit, residue, and distributed-agency relations.
StandaloneVECTOR	Depends on retained discrimination beyond prior PMS structures and same-burden rivals.

Dependency is not warrant inheritance

If node A depends on node B, defeating B can transmit vulnerability to A. The reverse does not follow: the existence or authority of B does not automatically warrant A. Dependency graphs therefore model declared vulnerability relations, not epistemic authority flow.

16. Computational representation, Reader boundary, integrity, and integration

PMS-VECTOR.yaml is a machine-readable repository reference. It is designed to support loading, inspection, case linkage, record scaffolding, declared graph generation, and consistency auditing. It is not an executable theory engine and does not convert syntax into substantive orientation, external factual warrant, legitimacy, or moral correctness.

16.1 What computational representation may check

Repository software may perform structural checks where the relation is explicitly declared. Such checks are useful because they expose representational drift without pretending to solve the underlying theoretical or empirical question.

Permitted structural check	Example
Required field presence	Whether a record carries fields required for its declared claim type.
Known construct status	Whether a status token belongs to the declared registry.
Known result type	Whether a stored result is recognized by the model.
PMS dependency resolution	Whether the required PMS Base reference is present/resolved.
Case index / case identity	Whether index entries and case files agree on identity.
Dependency target existence	Whether declared graph targets exist.
Provenance target existence	Whether claim/source identifiers referenced by provenance exist.
DIST core prohibition	Whether DIST has been accidentally reintroduced as a VECTOR core component.
ROT / RP distinction	Whether an operation family is being confused with stored audit state.
Type-firewall preservation	O/person worth, F/□, DIST/REV_avail, C_ROT/C_REV, Unknown/Available/Unavailable.
Constraint-firewall preservation	FixpointRequirement/BearerAuthority and Source(G)/Legitimacy(G).
E.23 reduction wording	Whether current same-burden status still cedes primary status where required.

16.2 What computation may not determine

No repository parser, schema checker, Reader, graph, VM scaffold, or simulation wrapper may claim to determine the following from VECTOR syntax alone:

- empirical truth or external factual warrant;
- moral correctness or legal validity;
- person evaluation, dignity, worth, or mature/adult status;
- actual actor understanding or uptake without evidence;
- actual effective capacity without evidence;
- a substantive DIR result merely from field presence;
- rival inferiority merely because the rival does not use VECTOR vocabulary;
- universal metric, total order, or universal confidence score;
- automatic legitimacy, authority, or obligation.

No automatic engines

The repository declares `direct_implementation_guaranteed = false`, `universal_metric_available = false`, `automated_direction_engine = false`, and `automated_moral_engine = false`. Computational form is an audit and integration affordance, not a claim that the model is algorithmically complete.

16.3 Operational use requirements

Any architecture-specific implementation that goes beyond inspection requires the PMS Base dependency to be resolved and must preserve the claim ceilings carried by the theoretical model.

Requirement	Meaning
Resolved PMS.yaml	Base semantics and governance are available.
Architecture-specific implementation	No universal executor is implied by the reference model.
Question-specific calibration where needed	Direction and sensitivity claims remain bounded to relevant calibration.
External warrant where material	Facts, measurements, and specialist relations remain externally supported.
Explicit claim ceiling	Outputs do not silently become stronger claim types.
Human/domain-grounded judgment where irreducible	Schema convenience does not erase substantive judgment.
Rival-sensitive evaluation	Implementations must preserve real winning conditions for external rivals.

VM or simulation scaffolding is a permitted repository implementation target only under explicit calibration. It is not a paper claim, does not imply validation, and does not imply theory expansion. A simulation can explore consequences of declared assumptions without turning those assumptions into warranted descriptions of the world.

16.4 Reader boundary

The planned VECTOR Reader is a **presentation, navigation, and repository-audit layer**. It may load folder or ZIP corpora, render Markdown/YAML, expose cases and references, and visualize declared repository relations. Its graphs are not an additional theoretical source.

Reader may visualize	Declared source
Architecture & Status	model/PMS-VECTOR.yaml
Dependency / Warrant Graph	reference/Dependency Map.yaml
Case Pressure Map	cases/index.yaml and case files
Selected Case Trace	Declared fields in the selected case record
Reduction Graph	Declared current status, case pressure, and reduction relations

Reader epistemic boundary

A Reader graph does not create theory, evidence, dependencies, authority, or classification. Visual adjacency does not create a dependency; node depth does not create ontological depth; centrality does not create theoretical importance. Only declared repository relations may be graphed as such.

The Reader must also preserve the Geometry Firewall. VECTOR is not to be visualized as a geometric vector space; ROT is not an angle; DIST is not spatial metric distance. The scientifically useful graph surface is the architecture's **dependencies, pressures, case traces, and reductions**, not a pseudo-geometric depiction of VECTOR itself.

16.5 Repository self-test boundary

A Reader or release self-test may identify internal repository inconsistencies: a case listed in the index but missing from disk, a dependency target that does not exist, a provenance target that cannot be resolved, or a reduction status that diverges across declared files. These checks are valuable because they test the integrity of the representation.

Consistency is not validity

reader self-test = repository consistency != VECTOR validity . A perfectly consistent repository can still contain a theoretically inadequate or empirically false claim. Conversely, a repository formatting error does not by itself refute the theoretical paper.

16.6 Required load order

For application or repository interpretation, the declared sequence preserves typed authority and prevents downstream files from becoming accidental sources of Base semantics.

Step	Source	Purpose
1	PMS.yaml	Resolve Base operator grammar, entry condition, and governance.
2	PMS-VECTOR.md	Read the primary VECTOR theoretical source.
3	model/PMS-VECTOR.yaml	Load the machine-readable VECTOR reference profile.
4	model/Model Specification.html	Interpret field contracts and representation boundaries.
5	reference/Claim Provenance.yaml	Resolve origin/status relations where needed.
6	reference/Dependency Map.yaml	Resolve declared vulnerability/dependency relations where needed.
7	cases/index.yaml	Load the paper-linked case inventory.
8	cases/*.yaml	Load individual case records.

Invalid shortcuts include using PMS-VECTOR.yaml before resolving PMS for application, treating a case result as context-free without the PMS and VECTOR model layers, or using a Reader graph as a substitute for declared source relations.

16.7 Release integrity contract

Repository release checks should verify that the representation still carries the enacted theoretical boundaries rather than merely parsing successfully. At minimum, the release should preserve the PMS dependency, Core/State distinction, DIST reduction, Geometry Firewall, external warrant boundary, LOGIC baseline firewall, uptake rule, system/person carriage boundary, provenance and dependency limits, Non-Capture, E.22 reduction link, and E.23 same-burden scope reduction.

Integrity relation	Required release condition
PMS dependency	Explicit; Base operators not redefined as VECTOR-owned.
Core / state	$\text{VECTOR} := \langle \text{DIR}, \text{ROT} \rangle ; \text{V}_t = \langle \text{DIR}_t, \text{RP}_t \rangle ; \text{ROT} \neq \text{RP}.$
DIST	Not in core; source relations preserved; $\text{DIST} \neq \text{REV}_{\text{avail}}.$
Human / operator firewalls	O/person worth, $F/\square$, DIR/∇ , ROT/Φ , $\text{ROT}/\text{PROJECT}_{\text{AS}}$, DIST/X , $\pi/\text{P}_{\text{MIP}}$, t/Θ remain distinct.
Option architecture	Unknown != Available/Unavailable; Invisibility != Blindness and is not itself a field error; Blindness/Illusion remain unsupported exclusion/inclusion; bounded-negative and Special-Case-Probe boundaries are preserved.
Attribution	Baseline LOGIC relation remains distinct from VECTOR orientation-mediated uplift.
Constraint	$\text{FixpointRequirement} \neq \text{BearerAuthority}$ and $\text{Source}(G) \neq \text{Legitimacy}(G).$
Calibration	Calibration failure maps to ClaimReduction or Suspend; notation does not manufacture warrant.
Tragic Constraint	Evidenced effective non-lossful route remains a falsifier.
Reduction	E.22 and E.23 links remain explicit; E.23 still cedes primary status for the tested class.
Reader	Reader authority remains prohibited.

16.8 Closing contract

The repository representation closes with the same compact architecture and reduction set carried by the final paper. These expressions are mnemonic summaries of typed relations, not independent formal axioms or metric operations.

Compact model identity
VECTOR := {DIR, ROT}
V_t(0 | F,Q,π) = {DIR_t, RP_t}
DIST := derived orientation-enactment separation descriptor

Domain	Compact non-equivalences
Direction	direction != destiny; local superiority != universal superiority; incomparability != equality
Perspective	rotation != relativism; probe != original-frame warrant
Time and possibility	distance != superiority; reversibility != reset; conceivable != available; availability != capacity; option transformation != unlimited agency
Consequence	cost != total score; unseen != invented
Normativity	capacity != obligation; constraint != legitimacy; necessity != moral goodness; self-binding != authority to bind others
Epistemic status	invariance != final truth; convergence != consensus; test-burden closure != truth closure
Human boundary	VECTOR != person ranking; VECTOR != dignity ranking; Adult Orientation shorthand != AdultPerson; VECTOR != tribunal

Additional governing boundaries
Being told is not uptake.
VECTOR robustness != external warrant.
Orientation↑ != Authority↑.
Provenance completeness != theoretical adequacy.
Dependency transmits vulnerability, not automatic warrant.
Self-application must preserve the possibility of self-reduction.

16.9 Final integration statement

This Model Specification is the human-readable bridge between the PMS Base dependency, the final VECTOR paper, and the machine-readable repository profile. It does not enlarge the theory. It makes explicit which artifact owns which semantics, how bounded claims are typed, where warrant and calibration remain external, how pressure can produce actual loss, how current reductions are represented, and which relations downstream software may inspect without converting repository consistency into theoretical authority.

The resulting model is intentionally capable of stopping with a directed relation, a trade-off, incomparability, collapse, suspension, revision, non-capture, domain deference, or theory reduction. Its strongest repository invariant is therefore not that VECTOR must survive every test, but that its representation must keep visible the conditions under which it may legitimately lose.

Model Specification stop condition
No downstream template, case file, provenance map, dependency graph, Reader, VM scaffold, simulation, render, or release artifact may silently strengthen the theory beyond the boundaries specified by PMS.yaml , PMS-VECTOR.md , and the aligned model/PMS-VECTOR.yaml . Where downstream artifacts expose a real contradiction or stronger rival, the proper response is revision or reduction of the affected VECTOR claim rather than reinterpretation of the artifact to preserve VECTOR.